



HOGESCHOOL ROTTERDAM / CMI

Besturingssystemen

TINBES03

Aantal studiepunten: 4

Cursusbeheerder: T. de Ruiter



Cursusbeschrijving

Cursusnaam:	Besturingssystemen
Cursuscode:	TINBES03
Aantal studiepunten en studiebelastinguren:	4 ec Dit studieonderdeel levert de student 4 ec op, hetgeen overeenkomt met een studielast van 112 uren waarvan 21 contacturen. De verdeling van deze 112 uren is als volgt: • 14 × 1,5 uur gecombineerd hoor- en werkcollege: 21 uur. • onbegeleid werken aan de practicumopdrachten: 30 uur. • onbegeleid werken aan de eindopdracht: 60 uur. • demonstreren eindopdracht: 1 uur.
Voorkennis:	• Computersystemen & Logica (TINCPS02-1) • Programmeren 1 (TINPRO02-1) • Programmeren 5b (TINPRO035B)
Werkvorm:	Gecombineerd hoor- en werkcollege
Toetsing:	Practicumopdrachten en eindopdracht
Leermiddelen:	• Een computer met een Linux-installatie (wordt in de eerste les behandeld). • Arduino Uno of Nano met USB-kabel. • Practicumhandleidingen, te vinden op Brightspace. • Boeken (niet verplicht): – Andrew S. Tanenbaum & Herbert Bos: <i>Modern Operating Systems</i>, 4th edition (2015). – Brian W. Kernighan & Dennis M. Ritchie: <i>The C Programming Language</i>, 2nd edition (1988). • Slides worden op Brightspace geplaatst voor de start van de les.
Leerdoelen:	De student: 1. Kan eenvoudig Linux systeembeheer uitvoeren met behulp van de command line. 2. Kan shell-scripts schrijven voor het automatiseren van eenvoudige taken. 3. Begrijpt hoe asynchrone input/output werkt en kan dit implementeren in een simpel besturingssysteem. 4. Begrijpt hoe een file system werkt en kan dit implementeren in een simpel besturingssysteem. 5. Begrijpt hoe dynamisch geheugenbeheer werkt en kan dit implementeren in een simpel besturingssysteem. 6. Begrijpt hoe processen uitgevoerd worden en kan dit implementeren in een simpel besturingssysteem. 7. Begrijpt hoe multi-tasking werkt en kan dit implementeren in een simpel besturingssysteem. 8. Begrijpt hoe prioriteit instellen werkt en kan dit toepassen. 9. Begrijpt hoe interprocescommunicatie (IPC) werkt en kan dit toepassen.
Inhoud:	In deze cursus wordt het onderwerp Besturingssystemen behandeld vanuit twee aspecten: het <i>gebruik</i> van een UNIX-achtig besturingssysteem en de <i>interne werking</i> van een besturingssysteem. Voor het eerste aspect ga je aan de slag met een serie opdrachten over Linux systeembeheer, shell scripting en interprocescommunicatie. Voor het tweede aspect ga je zelf een besturingssysteem implementeren voor de Arduino Uno, inclusief input/output (command line interface), bestandssysteem, geheugenbeheer en multitasking. Hiermee kun je meerdere programma's tegelijkertijd laten draaien op de Arduino, en data opslaan in bestanden. Daarnaast worden geavanceerde concepten behandeld die betrekking hebben op besturingssystemen, zoals semaforen, shared memory en prioritisering.
Opmerkingen:	De opdrachten moet individueel worden uitgevoerd.
Cursusbeheerder:	T. de Ruiter
Datum:	20 april 2026

1 Algemene omschrijving

1.1 Inleiding

Een besturingssysteem (OS: Operating System) is de belangrijkste software die op een computer wordt uitgevoerd. Het OS beheert alle software en hardware op de computer. Het regelt het geheugenbeheer, bestandsbeheer, de input/output, zorgt voor interrupts, voor het uitvoeren van meerdere onafhankelijke computerprogramma's op dezelfde computer, enzovoort. Zonder een OS is een computer nutteloos. Meestal zijn er verschillende computerprogramma's tegelijkertijd actief en ze moeten allemaal toegang hebben tot de CPU, het geheugen en de opslag van de computer. Het OS coördineert taken om ervoor te zorgen dat elk programma krijgt wat het nodig heeft. De drie meest gebruikte OS'en zijn Microsoft Windows, MacOS en Linux.

In deze cursus wordt het onderwerp Besturingssystemen behandeld vanuit twee aspecten: het *gebruik* van het Linux OS en de *interne werking* van een OS. Voor het eerste aspect ga je aan de slag met een serie opdrachten over Linux systeembeheer, shell scripting en interprocescommunicatie. Voor het tweede aspect ga je zelf een besturingssysteem implementeren voor de Arduino Uno, inclusief input/output (command line interface), bestandssysteem, geheugenbeheer en multitasking. Hiermee kun je meerdere programma's tegelijkertijd laten draaien op de Arduino, en data opslaan in bestanden. Daarnaast worden geavanceerde concepten behandeld die betrekking hebben op besturingssystemen, zoals semaforen, shared memory en prioritisering.

1.2 Relatie met andere onderwijseenheden

Besturingssystemen gebruikt de kennis die de studenten tijdens de cursus Computersystemen en de programmeer-leerlijn hebben opgedaan, met name Programmeren 1 (C) en Programmeren 5b (C++). Het begrip van de interne werking van het besturingssysteem ligt aan de basis van het gebruik van computers in de projecten, stage, TINLab en afstuderen.

1.3 Leermiddelen

- Een computer met een Linux-installatie (wordt in de eerste les behandeld).
- Arduino Uno of Nano met USB-kabel.
- Practicumhandleidingen, te vinden op Brightspace.
- Boeken (niet verplicht):
 - Andrew S. Tanenbaum & Herbert Bos: *Modern Operating Systems*, 4th edition (2015).
 - Brian W. Kernighan & Dennis M. Ritchie: *The C Programming Language*, 2nd edition (1988).
- Slides worden op Brightspace geplaatst voor de start van de les.

2 Programma

Het lesprogramma bestaat uit 7 weken, die elk gewijd zijn aan een thema. Binnen dat thema worden in de eerste les van die week aspecten behandeld die te maken hebben met Linux of OS'en in het algemeen. Bij ieder thema zijn één of meer opdrachten die beschreven staan in de **Practicumhandleiding Linux**. Deze opdrachten werk je tijdens de les of thuis uit in een readme op de cmi gitlab en worden door de docent afgetekend. In de tweede les van iedere week worden aspecten van het thema van die week behandeld die toegepast moeten worden in het zelf te bouwen OS voor de Arduino Uno of Nano (ArduinOS). De specificaties en instructies hiervoor staan in de **Practicumhandleiding ArduinOS**. Het besturingssysteem wordt geschreven in C of C++; hiervoor wordt eerst een aantal geavanceerde constructies herhaald, zoals pointers en structs. Voor het bestandssysteem wordt gebruik gemaakt van het EEPROM van de Arduino; dit wordt ook uitgelegd. Daarnaast worden geavanceerde concepten behandeld die betrekking hebben op besturingssystemen, zoals semaforen, shared memory, multitasking en prioritisering.

Voor de ontwikkeling van het OS is er een aantal *checkpoints*: stadia in de ontwikkeling die gevolgd moeten worden. We raden je nadrukkelijk aan om te zorgen dat aan het eind van iedere week het bijbehorende checkpoint behaald is, want als je eenmaal achterloopt is inhalen erg moeilijk. Je bent als student hier *zelf* voor verantwoordelijk.

Hieronder is het lesschema weergegeven, met voor iedere les de te maken opdrachten of het betreffende checkpoint waaraan gewerkt moet worden:



TI-Week (weeknr.)	Datum start week	Thema	Les	Onderwerpen	Opdrachten / Checkpoint
4.1 (17)	20-apr	Bootting up	1	wat is een OS? wat is Linux? wat is een kernel? virtual machines	1–3. installatie Linux (direct, in een VM of via WSL)
			2	blokdigram, herhaling C (pointers, strings, arrays, structs)	CP1: basisstructuur en ontwerp
(18)	27-apr			meivakantie	
4.2 (19)	4-mei	Input/output	3	BIOS, I/O devices, buses, hardware abstraction layer, synchronous vs asynchronous, half / full duplex, command line interface, shell, secure shell, pipes, redirecting	4. commando's 5. werken met textfiles
			4	non-blocking input, seriële buffer leegmaken, commando's, pointer naar functie	CP2: command line interface
4.3 (20)	11-mei	Bestandssysteem	5	bootloader, partitie, bestands-systemen (FAT, NTFS, APFS, ext2/ext3/ext4), Linux directory-structuur, journaling	6. bestandssysteem verkennen 7. werken met bestanden en directories 8. wildcards 9. eigenschappen (modes) 10. schijfruimte
			6	EEPROM, PROGMEM, FAT structuur, First Fit-algoritme	CP3: bestandssysteem
4.4 (21)	18-mei	Geheugen	7	fysiek en virtueel geheugen, geheugenbeheer, swapping, logical vs. physical addressing, mapping & paging, geheugenallocatie, endianness	11. shell scripts en variabelen
			8	memory table structuur, variabelen en typen, stack, getallen en strings pushen en poppen	CP4: geheugenbeheer, variabelen en stack
4.5 (22)	25-mei	Processen	9	geen les	
			10	process table structuur, processen starten en stoppen	CP5: processen
P (23)	1-jun			projectweek – geen les	
4.6 (24)	8-jun	Multitasking & scheduling	11	proces-toestanden, threads, Linux processen, multitasking, multiprocessing, scheduling (jobs, pre-emptive, round-robin, priority), real-time OS	12. processen bekijken 13. processen starten en stoppen 14. prioriteit instellen 15. scheduling
			12	bytecode-taal, round-robin processen uitvoeren, instructies uitvoeren	CP6: instructies uitvoeren
4.7 (24)	15-jun	Inter-process communication	13	semaforen, shared memory, signals, message queueing, named pipes	16. signals 17. message queueing
			14	flow control, timing, file I/O, forking	CP7: flow control
T (26)	22-jun				Demonstratie eindopdracht

3 Toetsing en beoordeling

3.1 Procedure

De toetsing bestaat uit drie onderdelen: het af laten tekenen van de Linux-practicumopdrachten, het demonstreren van de eindopdracht aan de docent en het inleveren van de broncode. Het is verplicht om het OS te schrijven voor een arduino UNO.

De uitwerking van de Linux-practicumopdrachten moet je in een readme plaatsen op de gitlab server van CMI, dit doe je binnen de groep op gitlab die is aangemaakt door de docenten. Hier krijg je een voldaan of niet voldaan, dit is een

voorwaarde voor het verkrijgen van de studiepunten. De opdrachten moeten uiterlijk in de week die volgt op de betreffende week afgetekend zijn.

Tijdens de toetsweek is er een demonstratie van je zelfgebouwde OS. Daar wordt je kort ondervraagd over de werking van je code (om vast te stellen of het je eigen werk is) en toon je aan op welke manier aan de leerdoelen voldaan is. Aan het eind van de toetsweek moet je de broncode van je OS inleveren op Brightspace.

3.2 Beoordeling

Voor ieder van de volgende aspecten wordt 0, 0.5 of 1 punt gegeven, als aan het aspect respectievelijk niet of nauwelijks, voor ongeveer de helft, of geheel is voldaan. De items 10–13 zijn bonusitems; deze zijn niet in de practicumhandleiding beschreven en bieden extra uitdaging. Voor deze bonusitems worden alleen punten toegekend als er met de basisitems 1–9 minstens 6 punten behaald zijn. Het eindcijfer is de som van de punten voor de verschillende aspecten. Het maximale eindcijfer is 10. Tussen haakjes staan de betreffende leerdoelen (zie p.2).

nummer	Criteria	Hoe kun je dit aantonen
1	Het OS leest opdrachten die op de command line worden gegeven (3).	Elk willekeurig commando van het OS uitvoeren.
2	Vanuit de command line kan data in bestanden worden opgeslagen, een lijst getoond van opgeslagen bestanden, de inhoud van bestanden getoond, bestanden gewist en de hoeveelheid beschikbare vrije ruimte getoond (4).	Een bestand aanmaken, uitlezen, verwijderen en het freespace commando uitvoeren
3	Er kunnen variabelen in het geheugen opgeslagen worden van de typen CHAR, INT, FLOAT en STRING, weer teruggehaald uit het geheugen, en van waarde veranderd (5).	Correct uitvoeren van het programma test_vars
4	Een bestand in het bestandssysteem kan als proces gestart, gepauzeerd en gestopt worden (6).	Het starten van een bytecode programma met een loop, deze pauzeren, hervatten en uiteindelijk weer stoppen
5	Er wordt van verschillende processen bijgehouden wat de toestand is (running, suspended, terminated), de waarde van de program counter, en welke variabelen bij dat proces horen (6, 7).	Het laten zien van de draaiende processen met de status, dit wordt ook gecontroleerd in de code
6	Het OS kan instructies in een bytecode-programma uitvoeren (6).	Indien er een bytecode programma wordt uitgevoerd kun je minstens een halve punt verdienen
7	Het OS kan meerdere processen tegelijk uitvoeren (afwisselend een instructie van elk proces) (7)	2 bytecode programma's met een loop tegelijk uitvoeren
8	Vanuit een bytecode-programma kan data van de typen CHAR, INT, FLOAT en STRING naar bestanden geschreven worden en uit bestanden gelezen (4, 6).	Het uitvoeren van de byteocde programma's: write_file en read_file
9	Vanuit een bytecode-programma kunnen andere processen opgestart (geforkt) worden en kan gewacht worden op het voltooien van een ander proces (6, 7).	Het uitvoeren van het bytecodeprogramma test_fork .
Bonus	Er zijn faciliteiten in het OS aanwezig voor het toekennen van prioriteiten aan processen, en een proces wordt uitgevoerd met de toegekende prioriteit (8).	Toelichten en demonstreren van eigen implementatie.
Bonus	Er zijn faciliteiten in het OS aanwezig voor het uitwisselen van data tussen twee processen door middel van shared memory (9).	Toelichten en demonstreren van eigen implementatie.
Bonus	Er zijn faciliteiten in het OS aanwezig voor het gebruik van semaforen voor gedeelde bestanden of shared memory (9).	Toelichten en demonstreren van eigen implementatie.
Bonus	Er is een compiler aanwezig die (een variant van) C- of Python-code omzet naar bytecode geschikt voor het OS (hoeft niet op Arduino te draaien; parser hoeft niet zelfgeschreven te zijn).	Toelichten en demonstreren van eigen implementatie.

3.3 Herkansing

De herkansing bestaat uit het opnieuw inleveren en demonstreren van de eindopdracht uiterlijk tijdens week 4.10. Ook moeten uiterlijk aan het eind van deze week de Linux-practicumopdrachten afgetekend zijn. Je moet hiervoor zelf een

afpraak maken met de docent.

4 Aanwezigheid

Aanwezigheid is verplicht bij de mondelinge toelichting van de eindopdracht, omdat hiermee vastgesteld wordt of de student eigen werk heeft ingeleverd.

5 Plagiaat

Onder plagiaat wordt verstaan: het overnemen van teksten, afbeeldingen of code zonder correcte bronvermelding. Hier vallen ook door kunstmatige intelligentie gegenereerde teksten, afbeeldingen of code onder, zoals ChatGPT. Je mag dus wel werk van anderen gebruiken in je eigen werk, als je maar duidelijk maakt dat het niet je eigen werk is en een correcte bronvermelding geeft. Plagiaat is een vorm van fraude. Onder fraude wordt eveneens verstaan als de student een andere student in de gelegenheid stelt onderdelen van zijn of haar werk over te nemen.

Het plegen van fraude en plagiaat is niet toegestaan. Ingeleverd werk kan gescand worden op plagiaat. Bij vermoeden van fraude en/of plagiaat wordt de Examencommissie geïnformeerd. Voor meer informatie zie de Hogeschoolgids bijlage 4, hoofdstuk 9.

6 Changelog

Studiejaar	Wijzigingen
2017–2018	eerste versie
2018–2019	wordprocessor-opdracht toegevoegd beoordeling verduidelijkt
2019–2020	opdrachten geïntegreerd tot 1 eindopdracht semaforen, shared memory en multitasking toegevoegd beoordeling herzien
2020–2021	aangepast aan on-line onderwijs afvinken checkpoints eruit gehaald bonusitems toegevoegd aan beoordeling
2021–2022	weer aangepast aan onderwijs op locatie memory table en process table over twee lessen verdeeld
2022–2023	Besturingssystemen 1 en 2 samengevoegd volgorde iets aangepast plagiaat-paragraaf toegevoegd
2023–2024	Uitzondering voor besturingssystemen 1 is niet meer geldig plagiaat-paragraaf uitgebreid
2024–2025	Verplichting gebruik gitlab toegevoegd beoordeling uitgebreid met uitleg hoe je het kan behalen.
2024–2025	data aangepast in programma

Bijlage: Toetsmatrijs

Leerdoel	Bloom-niveau	Toetsing
1	3	Practicum
2	3	Practicum
3	2, 6	Eindopdracht
4	2, 6	Eindopdracht
5	2, 6	Eindopdracht
6	2, 6	Eindopdracht
7	2, 6	Eindopdracht
8	2, 3	Practicum en eindopdracht
9	2, 3	Practicum en eindopdracht



Bloom-niveaus:

1. Remember
2. Understand
3. Apply
4. Analyse
5. Evaluate
6. Create